PROGRAMMING LOGIC AND DESIGN

PROG101 — INDIVIDUAL ASSIGNMENT 1

Commuter Transport System Logical Design for Sierra Leone

Student Name: Mamadu Alpha Bah

Student ID: 905006587

Course: PROG101 — Principles of Programming Logic and Design

Assignment Report

Table of Contents

- 1. Executive Summary & Abstract
- 2. Introduction & Local Context
- 2.1 Public Transport Challenges in Sierra Leone
- 2.2 Link to UN Sustainable Development Goal 11 (SDG 11)
- 3. Problem Decomposition & Algorithmic Design
- 3.1 Abstraction
- 4. System Logic & Input Validation
- 4.1 Input-Process-Output (IPO) Model
- 4.2 Algorithm & Pseudocode with Input Validation
- 5. Flowchart & Design Structure
- 5.1 Flowchart Symbols and Structure
- 5.2 Functional Modules & Sample Outputs
- 6. Design Rationale
- 6.1 Logic & Validation Rationale
- 6.2 Flowchart & Layout Rationale
- 7. Open Source Software & Digital Public Goods
- 7.1 Open Source Principles
- 7.2 Version Control & GitHub Deliverables
- 8. Conclusion & Reflection
- 9. References

1. Executive Summary & Abstract

When I started this assignment, my goal was to look at a real everyday issue we experience here in Freetown: unpredictable public transport fares, and design a simple logical algorithm to address it. Commuters taking poda-podas or shared taxis along busy routes such as Kissy Road, Lumley, or Regent often face sudden price increases during heavy traffic or peak rush hours.

To address this issue, I created a step-by-step logical design using pseudocode and a flowchart. The algorithm takes inputs such as distance zones, passenger counts, and peak traffic hours, then calculates a transport fare in Sierra Leone. It includes explicit input validation checks so that an invalid zone (outside Zones 1 to 4) or a passenger count exceeding vehicle seating capacity is rejected with an error message instead of producing an incorrect result. The work is intended to be uploaded step-by-step to a public GitHub repository in line with open-source and Digital Public Goods principles.

2. Introduction & Local Context

2.1 Public Transport Challenges in Sierra Leone

In Sierra Leone, especially in Freetown and surrounding towns, many people rely on commercial minibuses (poda-podas) and shared taxis to get to school or work. However, without a digital or standardized pricing system, passengers and conductors can end up arguing over charges. Fares may change depending on fuel costs, rainstorms, or heavy traffic around major junctions such as Congo Cross or Eastern Police.

By applying what I learned in PROG101, I designed a clear, transparent algorithm that calculates transport prices automatically based on set rules. An automated system can help protect passengers from being overcharged and help transport operators maintain clear boarding records.

2.2 Link to UN Sustainable Development Goal 11 (SDG 11)

UN Sustainable Development Goal 11 focuses on making cities safe, inclusive, resilient, and sustainable. Target 11.2 specifically calls for affordable and accessible public transport systems for everyone. This project aligns with that goal because transparent and publicly accessible transport-fare logic can help support fairer urban transport in local communities.

3. Problem Decomposition & Algorithmic Design

Trying to solve every issue in the transport system at once can feel overwhelming. I therefore used problem decomposition to divide the system into four manageable steps:

- Input collection and validation: request the distance zone, maximum vehicle capacity, number of passengers boarding, and peak-hour status, and verify that the values are valid.
- Base-fare calculation: assign a specific base fare according to the selected distance zone (Zone 1 through Zone 4).
- Surcharge application: check whether peak-hour conditions apply and add a percentage surcharge when required.
- Display results: print a clear summary showing the validated booking and total fare in Sierra Leonean leones.

3.1 Abstraction

To keep the logic focused, I used abstraction to leave out details that do not directly affect ticket pricing, such as vehicle engine types, driver names, or fuel brands. The design focuses on the main variables that affect the calculation: distance zone, peak travel time, vehicle seating capacity, and passenger count.

4. System Logic & Input Validation

4.1 Input-Process-Output (IPO) Model

I structured the system using the Input-Process-Output (IPO) model to make the logical flow clear.

Input	Processing	Output
Distance zone (1 to 4)	1. Validate zone (must be 1, 2, 3, or 4).	Booking confirmation
Vehicle capacity	2. Validate capacity (> 0).	Validated passenger count
Passenger count	3. Validate passengers (> 0 and ≤ capacity).	Final fare (in leones)
Peak-hour flag (Yes/No)	4. Calculate base fare from zone.	
	5. Apply 20% peak surcharge if applicable.	

4.2 Algorithm & Pseudocode with Input Validation

The following pseudocode includes validation loops to reject invalid distance zones and vehicle overcrowding.

```
MainProgram:
    Start
    Declare zone, passengers, capacity As Integer
    Declare isPeak As String
    Declare fare As Real

    // Validation 1: Distance Zone Input (Zones 1-4)
    Display "Enter Distance Zone (1: Short, 2: Medium, 3: Long, 4: Express):"
    Input zone
    While (zone < 1 OR zone > 4)
        Display "ERROR: Invalid distance zone (1-4). Re-enter:"
        Input zone
    EndWhile

    // Validation 2: Vehicle Capacity Input
    Display "Enter Maximum Vehicle Seating Capacity:"
    Input capacity
    While (capacity <= 0)
        Display "ERROR: Capacity must be > 0. Re-enter:"
        Input capacity
    EndWhile

    // Validation 3: Passenger Count Input
    Display "Enter Number of Passengers Boarding:"
    Input passengers
    While (passengers <= 0 OR passengers > capacity)
        If passengers > capacity Then
            Display "ERROR: Overcrowding detected!"
        Else
            Display "ERROR: Invalid passenger count."
        EndIf
```

```
        Display "Re-enter valid passenger count:"
        Input passengers
    EndWhile

    Display "Is it peak hour? (Yes/No):"
    Input isPeak
    fare = CalculateFare(zone, isPeak)

    Display "========================================="
    Display "TRANSIT TICKET SUMMARY"
    Display "Zone: " + zone + " | Passengers: " + passengers + " / " + capacity
    Display "Total Fare: Le " + fare
    Display "========================================="
    End

Function CalculateFare(zone, peak) As Real
    Declare baseFare, totalFare As Real
    If zone == 1 Then
        baseFare = 5.0
    ElseIf zone == 2 Then
        baseFare = 8.0
    ElseIf zone == 3 Then
        baseFare = 12.0
    ElseIf zone == 4 Then
        baseFare = 18.0
    EndIf

    If peak == "Yes" OR peak == "yes" Then
        totalFare = baseFare * 1.20
    Else
        totalFare = baseFare
    EndIf
    Return totalFare
EndFunction
```

5. Flowchart & Design Structure

5.1 Flowchart Symbols and Structure

- Start (oval): initialize system variables for route ID, capacity, and passenger count.
- Input (parallelogram): enter RouteID, MaxCapacity, and initial PassengerCount.
- Decision (diamond): check whether PassengerCount is less than MaxCapacity.
- If No: display “Vehicle Full: Boarding Closed” and terminate the process.
- If Yes: proceed to enter the distance zone and peak-hour status.
- Processing (rectangle): call the CalculateFare function, calculate the base fare with any peak surcharge, and increment passenger count by one.
- Output (parallelogram): display the ticket summary and updated passenger manifest.

5.2 Functional Modules & Sample Outputs

Scenario input: Route ID “FRT-04”, Distance Zone 2 (Medium), Peak Hour “YES”, initial passenger count 13, and maximum capacity 14.

The source example states that the base fare for Zone 2 is Le 10.00 and uses a 1.25 peak multiplier, giving a final fare of Le 12.50. The passenger count changes from 13 to 14.

System display messages:

- Boarded Successfully. Fare: Le 12.50
- Current Count: 14 / 14
- WARNING: Vehicle Full. Boarding Closed.

6. Design Rationale

6.1 Logic & Validation Rationale

To support system accuracy, the algorithm checks input data before processing it.

- Zone validation rationale: allowing arbitrary zone numbers could cause calculation errors or unfair default pricing. A While loop around zone entry requires the user to enter a number from 1 to 4.
- Capacity validation rationale: commercial vehicles have passenger limits. Allowing passenger counts above vehicle capacity can create safety risks and overcrowding. The validation logic compares passenger count with capacity and refuses to continue until valid values are supplied.

6.2 Flowchart & Layout Rationale

The flowchart is arranged so that arrows move from top to bottom and left to right without crossing. When an invalid entry occurs, the decision diamond sends a reject arrow back to the relevant input box. This loop makes the process easier to follow and helps ensure that invalid inputs cannot bypass verification.

7. Open Source Software & Digital Public Goods

7.1 Open Source Principles

I chose to document this project transparently using an open-source approach. Making the programming logic open source allows other students, local developers, or transport managers in Sierra Leone to adapt the algorithm for their own town or route and reuse it without proprietary software restrictions.

7.2 Version Control & GitHub Deliverables

To track progress and demonstrate how the project was developed step by step, I plan to maintain a public repository on GitHub.

- Repository Name: PROG101_Assignment1_MamaduAlphaBah_905006587
- Repository URL: To be added after creating the GitHub repository.

Commit history summary:

- Initial project setup and README.md creation
- Pseudocode and flowchart design
- Final project report PDF

8. Conclusion & Reflection

Working on this assignment helped me see how programming logic applies to everyday life in Sierra Leone. By taking a common problem—confusion about poda-poda fares—and applying computational thinking, I developed a structured algorithm with input validation and decision loops. Learning to document code, design flowcharts, and use version control on GitHub provides a foundation for future programming modules.

9. References

- Digital Public Goods Alliance. (2025). Standard for Digital Public Goods. DPGA Governance Publications.
- Farrell, J. (2018). Programming Logic and Design, Comprehensive (9th ed.). Cengage Learning.
- Fullah, E. (2026). PROG101 Principles of Programming Logic and Design: Individual Assignment Brief. Department of Information and Communication Technology, Limkokwing University of Creative Technology, Sierra Leone.
- Open Source Initiative. (2026). The Open Source Definition. Available online: https://opensource.org/osd
- United Nations. (2015). Transforming our world: The 2030 Agenda for Sustainable Development (Goal 11: Sustainable Cities and Communities). New York: UN Department of Economic and Social Affairs.